appliedAI Developers | codecentric | ZenML labs

Welcome



Program starts at 18:30

Follow us on Linkedin

appliedAI institute for europe

appliedAI Developers

Engineers, researchers, and tech leaders solving real-world production AI challenges.
We run techie-to-techie meetups to share hard learned lessons and what actually works when shipping AI at scale.



What we value?

- making AI development clear
- connecting technical stories

with real impact

Get involved:

- share your lessons learned & challenges
- network with inspiring developers

Grow with us:

- host a meetup
- share the knowledge
- expand your skill set

Follow & Contact us

appliedAI institute for europe

Disclaimer

appliedAI Developers

By attending this event today, you grant the organizers the full right to use the images resulting from photo/video recordings, as well as any reproductions or adaptations of those images, for promotional purposes. This may include (but is not limited to) the right to use them in print and online advertising, and on social media. Please contact us if you have any questions.

appliedAI institute for europe

Agenda

appliedAI Developers

18:30 – 18:50	Arrival & mingle
18:50 – 19:00	Opening: framing the core question
19:00 – 19:20	👩‍💻 Talk: Alex: Observability as the foundation of trustworthy AI
19:20 – 19:50	🍕 Slice-breaker + Q&A
19:50 – 20:05	👩‍💻 Talk #2: Alexej: From traces to better agents
20:05 – 20:55	🧩 World Café Challenge
20:55 – 21:00	Closing: observability, synthesised
21:00 – 22:00	Networking 🍕

appliedAI institute for europe

appliedAI Developers

Why this series exists

Most AI events focus on:

- models
- tools
- demos
- benchmarks

Nexus focuses on:

- production failures
- architectural patterns
- tradeoffs
- what actually breaks after the demo works

Goal: build a production readiness blueprint for agentic systems.

appliedAI Developers

Production Readiness Blueprint

1. Context — what the system sees
2. Behavior — how it decides
3. Execution — how it acts
4. State — what it remembers
5. Collaboration — how humans stay in control
6. Observability
7. Security
8. Learning & Adaptation
9. Scaling, Performance & ROI

appliedAI institute for europe

Registration Signal

appliedAI Developers

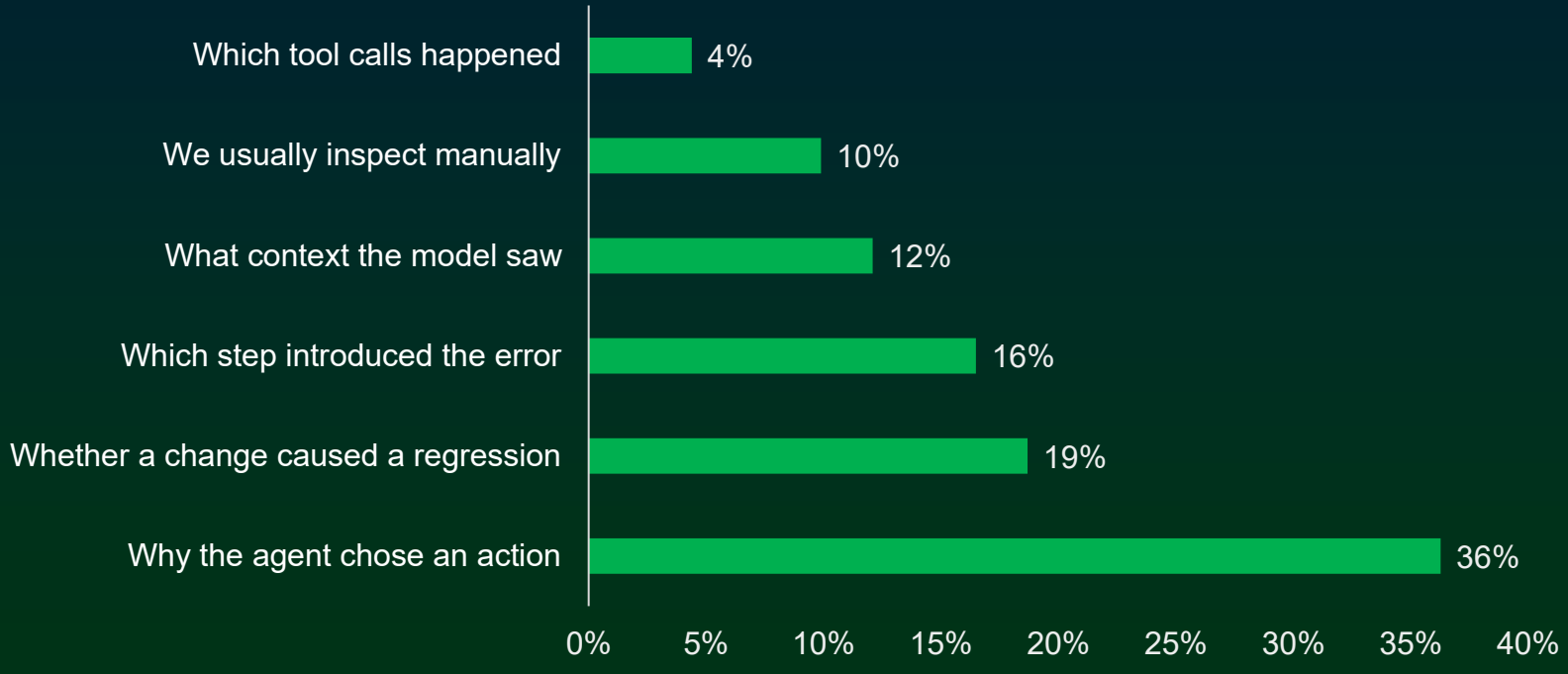
Attendee profile

- AI and agent engineers
- Software architects
- Observability and governance specialists
- Researchers, students, and engineers entering the field
- Practitioners working on RAG, evals, and agent harnesses
- CTOs, founders and startup builders

appliedAI institute for europe

appliedAI Developers

What is hardest to reconstruct?



appliedAI institute for europe

appliedAI Developers

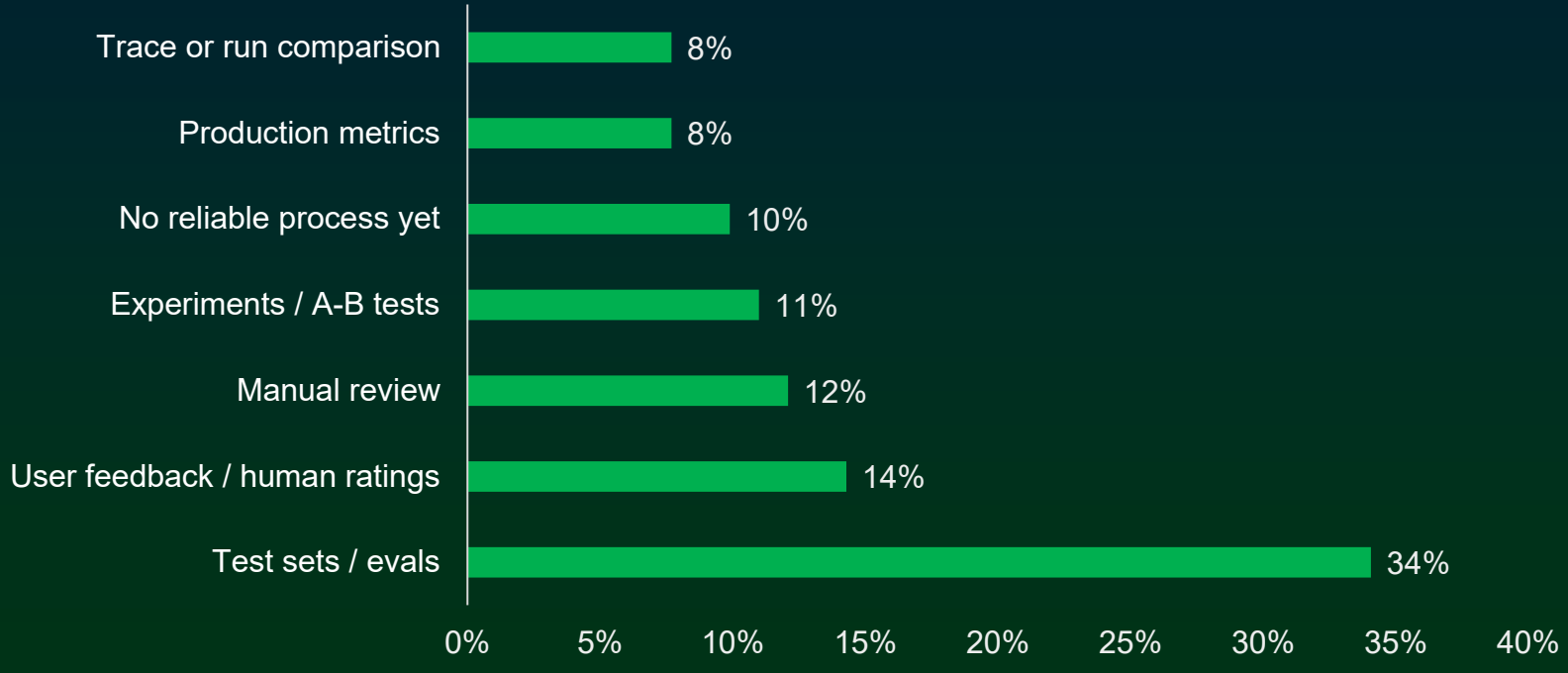
What failure is hardest to catch early?



appliedAI institute for europe

appliedAI Developers

How do you currently judge improvement?



appliedAI institute for europe

appliedAI Developers

What the written responses reveal

1. Silent correctness failures:
systems returning plausible, well-formed results that were factually wrong or failed the real task. Logs and dashboards did not signal a problem.

2. Incomplete evidence, even when tracing exists:
Missing input context, incomplete traces, hidden prompt changes, and unreliable decision reconstruction.

3. Metrics improve while actual quality deteriorates:
small evaluation sets, changing retrieval rankings, green metrics but user complaints.

4. Human judgment is difficult to scale:
Reviewers lack original context, relevant domain expertise, or evaluation framework.

appliedAI Developers

When AI fails, the system often looks healthy.

1. **Observability tools can give a false sense of completeness**
2. **The agent can be wrong when every metric is green**
3. **Correct conclusions and correct reasoning are different things**
4. **Human review is not automatically a reliable ground truth**

Question for tonight:

How do we know our measurements represent reality rather than simply confirming the assumptions we built into them?

appliedAI Developers

Alexander Machado

appliedAI institute for europe

Observability

Observability as the foundation of Trustworthy AI



Alexander Machado · Head of Trustworthy AI · appliedAI Initiative

appliedAI Initiative



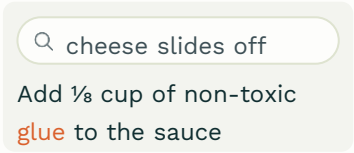
Mentimeter

menti.com | ID: **1119 4886**

01 · THE PROBLEM · WHAT THE CASES SHARE

When an AI agent fails, can you explain why?

Wrong answer

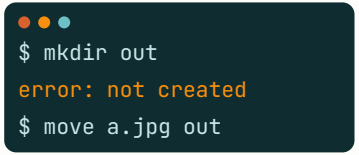


Google AI Overviews

Glue on pizza, sourced from a Reddit joke.

Trace the sources, not just the output.

Wrong path

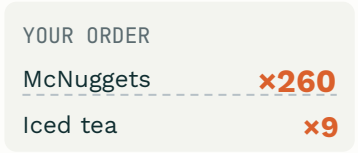


Gemini CLI

A failed mkdir went unchecked, then files were overwritten.

Record real tool results, not beliefs.

Oversight



McDonald's drive-thru

Wrong orders went straight through to customers.

Check before actions reach people.

Accountability



Air Canada chatbot

Invented a refund rule. The airline had to pay.

You own every answer. Keep the evidence.

KEY TAKEAWAY

Without evidence, nobody can explain what the agent saw, decided and did.

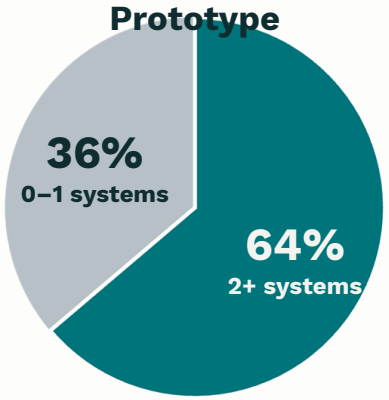
Prototyping agents is fast. Deploying trustworthy agents to production is hard.

01 · THE PROBLEM · PROTOTYPE VS PRODUCTION

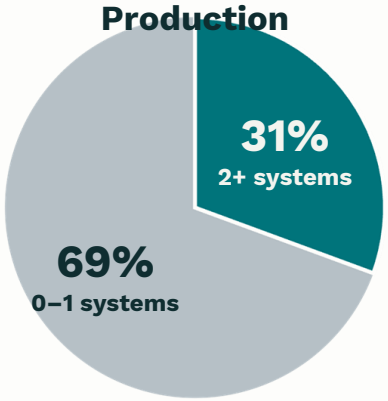
Lots of prototypes, few agents in production

Experimentation is strong. Conversion to production stalls.

Agentic AI systems in **Prototype**



Agentic AI systems in **Production**



Companies with 2+ systems

64% → 31%

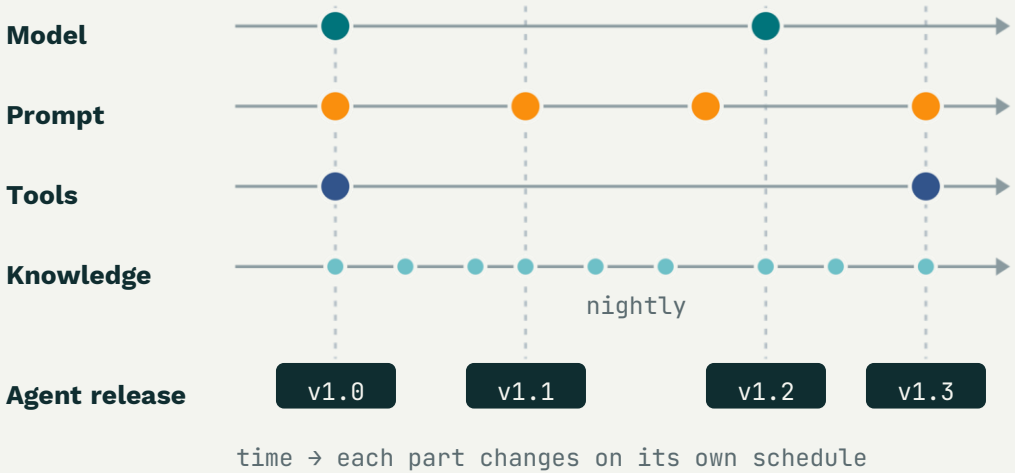
Two in three run several prototypes. Fewer than one in three get several agents live.

The gap is integration, adoption and trust.

Source: appliedAI Initiative Ecosystem Meetup, AgentOps Day 2025 · share of companies by number of agentic AI systems per stage

01 · THE PROBLEM · VERSIONING

Agents change weekly. Can you answer these?



What production will ask you

Which **model and prompt** answered this customer last Tuesday?

Which **knowledge base** version did v1.2 retrieve from?

What was the success rate of v1.1, before the **tool** change?

Which agent versions were **live** in the last 3 months?

Every trace must carry the versions behind it: model, prompt, tools and data.

01 · THE PROBLEM · AT SCALE

Production raises new questions

Imagine we roll out our customer-support agent from one market to five.

Traceability

Why did it skip the knowledge base and answer from memory?



Versioning

Which versions answered this customer last Tuesday?

```
model   v3.1 → v3.2
prompt  v12  → v14
tools   4 → 6 tools
kb      re-indexed nightly
```

Evaluation

The new model scores better overall. What got worse?



Oversight

Who reviews the agent before it issues a refund on its own?



All four depend on one foundation: observability

02 · THE SOLUTION

Trustworthy AI runs on evidence

DEFINITION

Trustworthy Agent Engineering

An integrated approach: one framework to design, govern, observe and scale agentic AI in a way that is lawful, ethical and robust, minimizing risks like harm to people.

Design **Govern** **Observe** **Scale**

Observability is the engineering enabler: it produces the evidence every principle depends on.

Principle · the question observability helps answer

Transparency
What happened inside the system?

Explainability
Why did it produce this outcome?

Accountability
Who is responsible, and what evidence supports the decision?

Reliability & robustness
Does it behave consistently under expected and unexpected conditions?

Human oversight
Can a human understand the situation and intervene?

From principles to practice: the agentic AI lifecycle

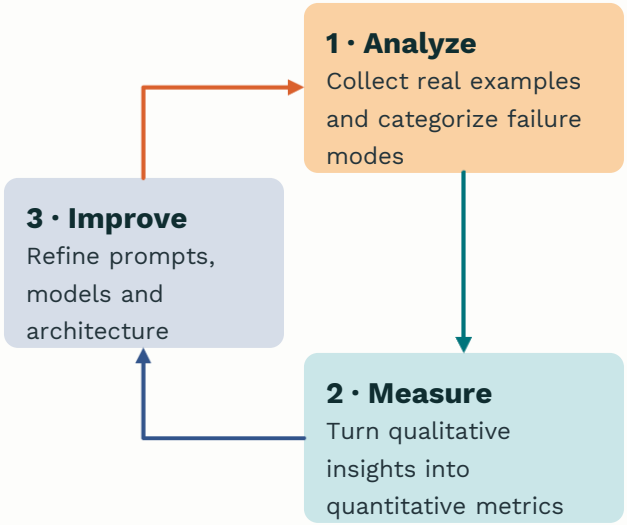
We embed the principles across the whole lifecycle. Observability is an important part of it.



initiative for applied artificial intelligence

What are evals?

Evals are the systematic measurement of quality in an LLM or agent pipeline. A good eval gives results that are easy and unambiguous to interpret: usually a score, sometimes a qualitative report.



Reference-based evals

Compare the output with a known ground-truth answer.

- Output matches the reference string, fully or partly
- Run the SQL and check the result matches the known answer

Reference-free evals

Judge the output on its own properties or rules, with no golden answer. Used for subjective outputs.

- Is this summary faithful and useful?
- No toxic language in a chatbot
- Generated SQL runs without syntax errors

03 · OBSERVE & EVALUATE · TWO LOOPS

Two loops, two kinds of evals

In agentic AI, online evals matter most: they find the failures you couldn't predict.



Offline evals before release

- Start with ~100 cases, grow to hundreds
- Regression gates and red-teaming before go-live
- Compare versions case by case, not just the score

Online evals **Key for agentic AI**

- Easiest start, highest return. Needs tracing
- Under 10 metrics per use case + a health index
- Calibrated LLM judges and real user signals

1 Develop · build & instrument → **2 Test** · trace runs → **3 Deploy** · version & gate → **4 Operate** · monitor → **5 Improve** · feed back

Measure what matters to your users

Best practice: error analysis

01 **Collect real user interactions** → 02 **Label freely what went wrong** → 03 **Cluster problems into categories** → 04 **Measure and track those categories**

Avoid generic metrics

~~hallucination rate~~ ~~toxicity~~

They don't reflect real end-user outcomes.

Application-specific KPIs

- How many turns until a customer successfully changes their address?
- Does the agent disambiguate ambiguous inputs?

Platform-level metrics

Apply across all agents, for example:

- Never mention competitors
- Always respond in the customer's language

04 · THE METRICS · TRUST PRINCIPLES

Do your KPIs cover the trust principles?

KPIs mapped to trustworthy AI principles

Business *executives*

Efficiency → *Value*
Cost per run **5** · Per-model token usage **3** · Operational cost **2**

Task performance → **Reliability**
Process optimization results **3** · Goal completion

Adoption → *Value*
Active monthly users · Employee AI training rate **1**

Compliance *governance*

Safety → **Reliability**
Reported incidents **3** · Policy violations · Guardrail triggers

Human oversight → **Human oversight**
Accountability per agent **3** · Escalation and override rate

Reproducibility → **Accountability** → **Transparency**
High-risk systems identified **3** · Requirements completed **2** · Trace completeness

Technical *engineers*

Task performance → **Reliability**
First-pass success rate **4**

Efficiency → *Value*
Time to first token **2** · Throughput **1** · Latency

Reliability → **Reliability**
Impact-weighted error rate **1** · Tool errors · Retries

Quality → **Explainability**
Trajectory evals · Groundedness · Evaluator scores

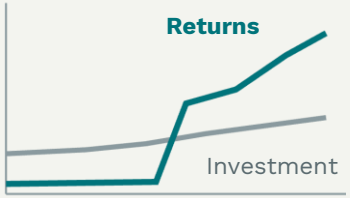
AI monitoring protects revenue, not just accuracy

Every number below comes from production traces. **AgentOps provides the ground truth.**

Use case dashboard

AgentOps tracks performance across the lifecycle: task success, completed workflows, failed tasks, escalation rate.





Tracking success rates

Metric	Value
Task success rate	**82%**
Escalation to human	**18%**
Average response time	**4 s**

€5 of human work per automated task. 100k tasks: **€410k**.

Linking accuracy to failure

Metric	Value
Failed tasks	**6,000**
Cost per human recovery	**€8**

6k failed tasks × €8 = **€48k**. Fewer failures, better ROI.

Measuring productivity gains

Metric	Before AI	After AI
Avg processing time	12 min	**2 min**
Tasks/day per employee	40	**120**

Turn the gain into labor savings or capacity.

Linking version performance to business impact

AI accuracy	Auto-resolved	Monthly savings
70%	35,000	**€140k**
82%	41,000	**€176k**
90%	45,000	**€200k**

05 · TRACEABILITY · CAN AND CANNOT

What a trace can and cannot tell you

Traceability is the ability to reconstruct, step by step, what an agent did: every input, prompt, retrieval, tool call, decision and output, linked to the versions behind them, in one trace you can replay.

A trace can establish

- What was retrieved, and when
- Which model, prompt, tool and config versions ran
- Which intermediate decision led to which action

What remains uncertain

- Why the model weighted one source over another
- Whether its stated reasoning reflects what drove the output
- Whether a slightly different input would change the outcome

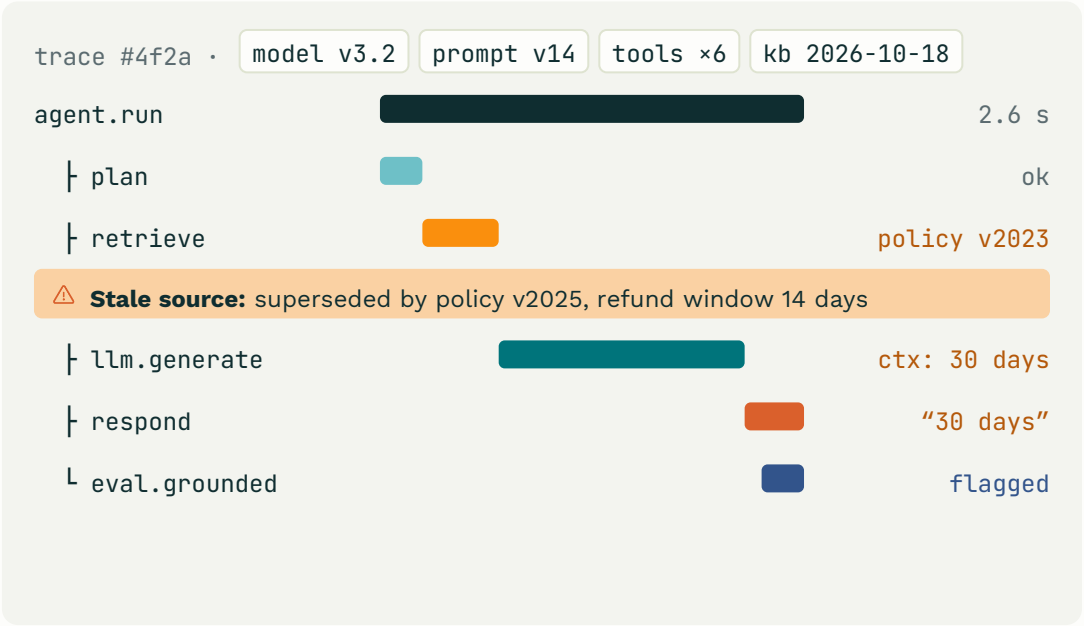
When human judgment is needed

- High-impact or irreversible actions
- Conflicting or incomplete evidence
- Situations outside the evaluated scenarios

05 · TRACEABILITY · IN PRACTICE

The refund case, traced

Remember the agent that promised 30 days instead of 14? One trace shows where it went wrong.



OBSERVABILITY → evidence

Root cause found

Retrieval returned an outdated policy. Generation did its job.

EVALUATION → signal

Claim not grounded

A check compares “30 days” with the current policy and flags it.

OVERSIGHT → decision

Human corrects

Reviewer fixes the answer, re-indexes the KB, adds a regression test.

What a trace would have shown

Wrong answer

Google AI Overviews

What people saw

Glue on pizza

```
The trace shows
retrieve → Reddit
11-year-old joke
```

Filter satire and forum sources

Wrong path

Gemini CLI

What people saw

Files gone

```
The trace shows
mkdir → error
agent: "success"
```

Check every tool result

Oversight

McDonald's drive-thru

What people saw

Viral wrong orders

```
The trace shows
McNuggets ×260
confirm step: none
```

Gate unusual orders before they go through

Accountability

Air Canada chatbot

What people saw

A tribunal ruling

```
The trace shows
page: no retroactive
answer: 90 days ok
```

Check answers against the source

Each failure sat in one step. You only find it if you trace every step.

Illustrative traces reconstructed from public reporting, not actual system logs

06 · CONCLUSION

Observability as the foundation for trustworthy agents

Debug faster

Find the source of failures instead of guessing.

Explain behavior

Reconstruct what happened and why, within the limits of available evidence.

Support accountability

Maintain evidence for review, incident analysis and audit.

Improve continuously

Turn production observations into better evaluations and safer releases.

06 · CLOSING QUESTION

If your agent makes a high-impact decision tomorrow, would you have the evidence to explain it?



Thank you

Alexander Machado · Head of Trustworthy AI

Scan to connect on LinkedIn

appliedAI Initiative

Break

From Traces to Better Agents

Alexej Penner

Founding Engineer, ZenML

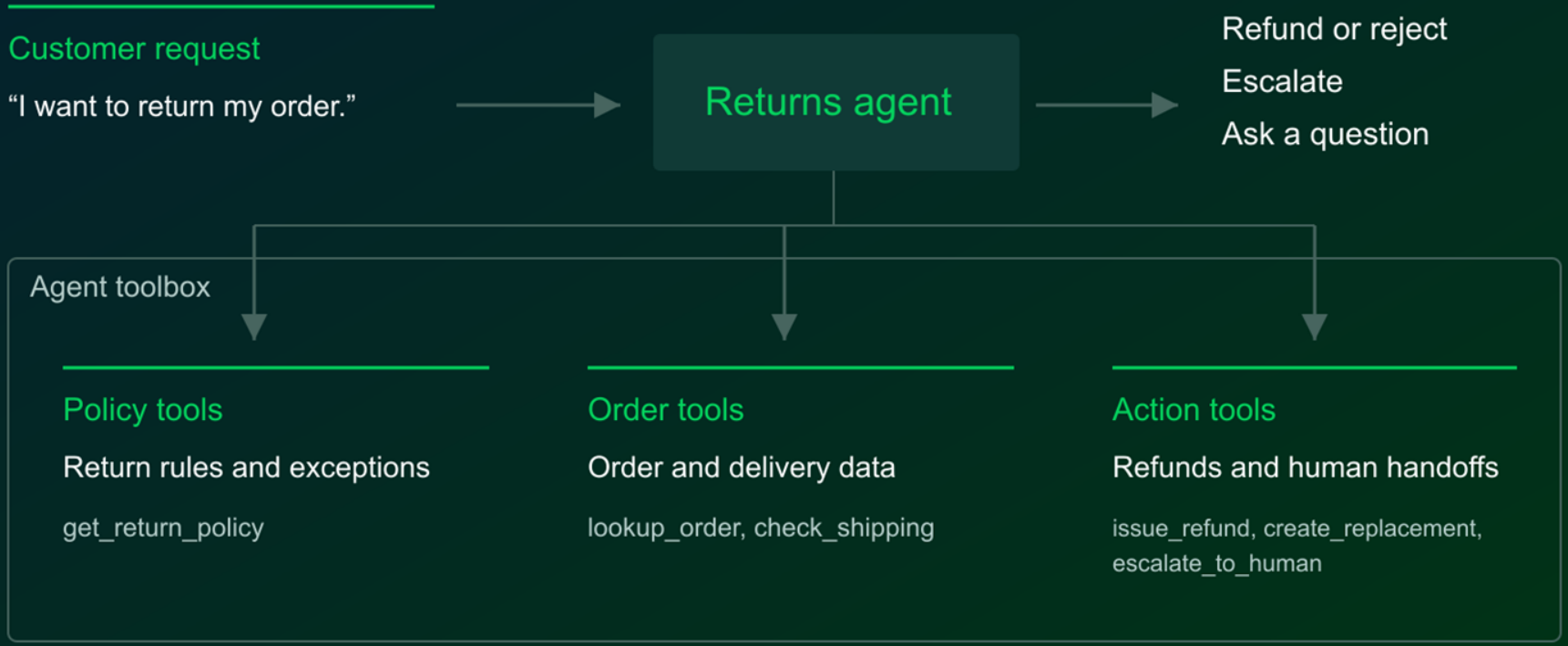
ZenML Labs

Building tools for ML and agent engineering

Your agent is in production.

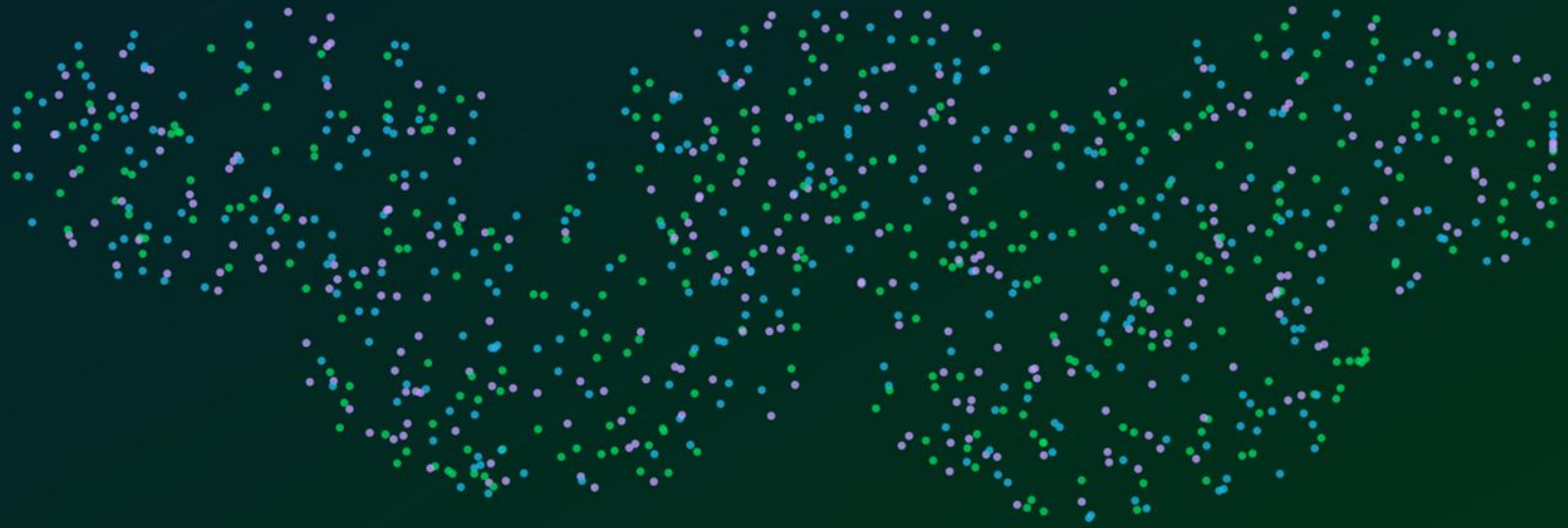
How do you use what happens there to make the next version better?

Our example agent



Millions of return sessions

- Refund issued
- Rejected
- Escalated



Which sessions should we investigate?

Refund issued, item value > €500



Session investigation

Policy: standard returns within 30 days. Damaged replacements may qualify for an exception.

SESSION A

Headphones

€640

Return reason

Changed my mind

Since delivery

12 days

Refund issued

SESSION B

Monitor

€920

Return reason

Damaged replacement

Since delivery

45 days

Refund issued

SESSION C

Camera

€780

Return reason

Changed my mind

Since delivery

45 days

Refund issued

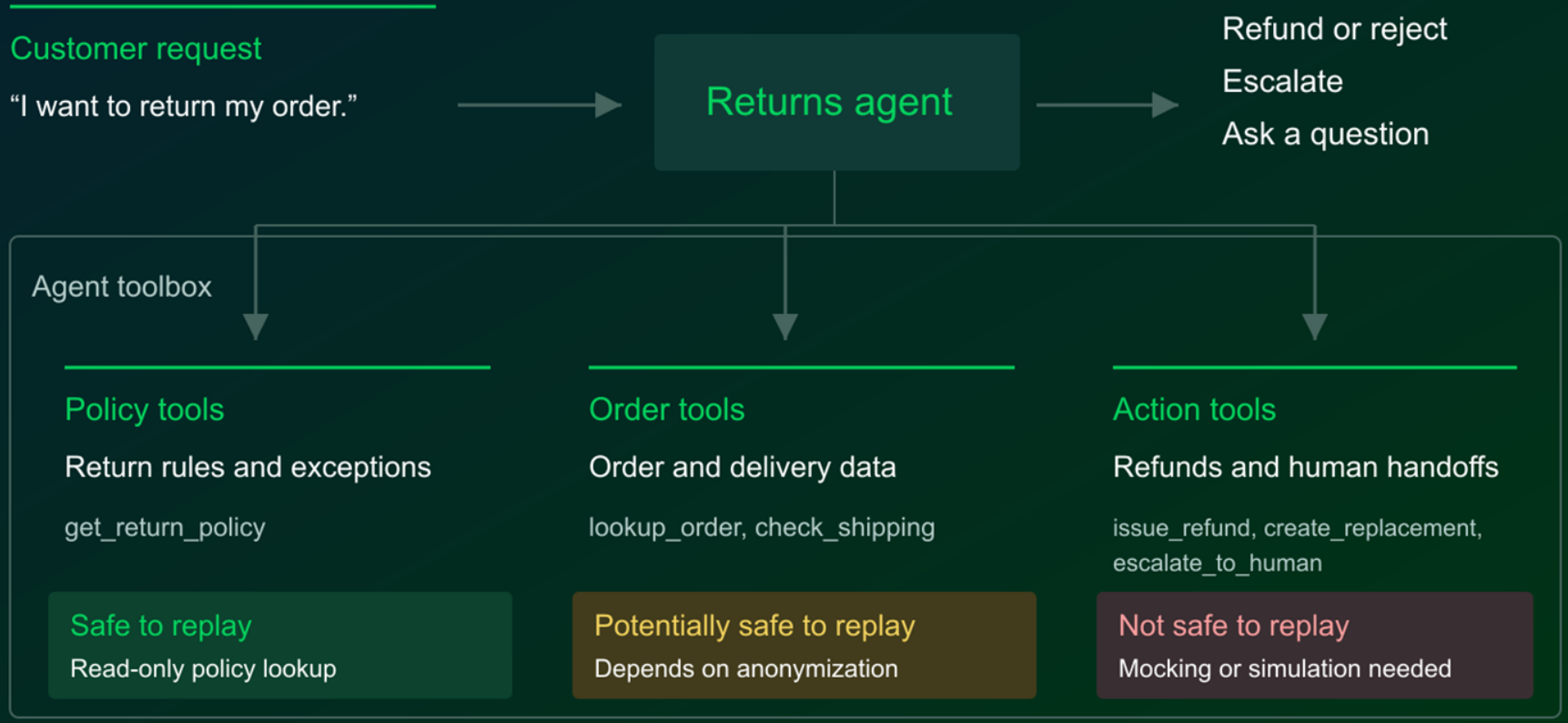
Trace investigation

Test sets for different problems

- Golden dataset
- Policy edge cases
- Complex queries



Tools in the test environment



Tool behavior during replay

Approach	What happens	Advantage	Tradeoff
history Recorded result	Match tool + arguments. Return the saved result.	Reuse captured behavior. No tool execution.	New arguments can miss. Recorded coverage matters.
static Configured result	Match configured cases. Return a chosen result.	Control exact conditions. No tool execution.	Write and maintain cases. Responses stay scripted.
passthrough Execute tool	Run the tool code. Use its current result.	Exercise real code. Support different paths.	Prepare the environment. Side effects happen.
simulated LLM response	A model generates the tool response.	Handle new calls. Explore different paths.	Validate the simulator. Added cost and variability.

Preparing production traces for testing

Data	Preparation	Effect on tool policies
Names, emails, IDs	Use consistent replacements across messages, tool arguments, and results. Keep the reason for the return.	history Update recorded calls too, so arguments still match.
Dates and amounts	Preserve delivery age, currency, and approval limits. Shift dates and the test clock together.	static / simulated Keep fixtures and generated responses consistent.
Production references	Map orders and customers to records in your test environment.	passthrough Tools must resolve the test IDs.

Model experiment

	Original	Replay
Model	GPT-6 Luna	Qwen3.8-2.4T-A95B
System prompt	Apply the return policy. Resolve the request using the available tools.	Apply the return policy. Resolve the request using the available tools.
Tools	lookup_order, issue_refund, ...	lookup_order, issue_refund, ...
Evaluator	returns-quality, ...	returns-quality, ...
Test cases	Policy edge cases, ...	Policy edge cases, ...

System prompt experiment

	Original	Replay
Model	GPT-5 nano	GPT-5 nano
System prompt	Favor a generous refund. Assume action tools enforce approval limits.	Check policy and approval limits. Escalate when approval is required.
Tools	lookup_order, issue_refund, ...	lookup_order, issue_refund, ...
Evaluator	returns-quality, ...	returns-quality, ...
Test cases	Policy edge cases, ...	Policy edge cases, ...

Experiment results

	Baseline GPT-5 nano	Model change GPT-4.1 nano	Prompt change GPT-5 nano
Quality score	0.502	0.410	0.792
Mean cost per session	$0.000962	$0.000316	$0.001741
Latency	…	…	…

The next step: production rollout

Candidate

Small share of traffic

Production feedback

Correctness
Escalations
Cost and latency

Decision

Expand
Roll back

New traces feed the next investigation

Let’s look at an example of this

Returns resolver: traces, cohorts, and experiments

Ingredients for a robust outer loop

01

Production evidence

Inputs, tool calls, outcomes and the context behind them

02

(Human?) judgment

What should have happened?
A verdict grounded in evidence

03

Cases that challenge the change

The failure you want to fix
Cases that already work
Unseen cases to catch overfitting

04

Evaluators that measure the behavior

Turn reviewed judgments into checks
Score actual actions and outcomes
Validate against human review

05

Controlled experiments

One change, fixed inputs
explicit tool behavior
quality, cost and latency

06

Rollout and feedback

Gradual exposure and rollback
new evidence for the next cycle

Every result stays linked to its cases, configuration and evidence

appliedAI Developers

Challenge

Design the Observability Loop for Agent Improvement

appliedAI institute for europe

appliedAI Developers

Scenario

Your AI system analyzes 100,000+ Jira issues to discover recurring defects, root causes, and emerging patterns.
It identifies a previously unknown relationship between incidents.

The finding is plausible and supported by several issues. But it isn't in your golden set, and domain experts cannot easily verify it without investigating the underlying evidence.
Meanwhile, traditional evaluations sometimes mark valid results as wrong because the model found different evidence or explanations.

Your challenge: Design an observability and evaluation approach that can distinguish genuine discoveries from convincing hallucinations — without requiring humans to review everything.

step	prod (gpt-mini)	move	trade vs prod
categorize	92% ✓	= keep gpt-mini	cheapest gate-passer
completeness	85% ✗ schema	↑ gpt-full	+15% quality · +242% $/call
subtasks_planner	94% ✓	= keep gpt-mini	cheapest gate-passer
plan_task	78% ✓	= keep gpt-mini	cheapest gate-passer
search_plan	76% ✗ quality	↑ gpt-full	+24% quality · +244% $/call
evaluate_retrieval	80% ✗ quality	↑ gpt-full	+20% quality · +268% $/call
deep_investigate	65% ✗ quality	↑ gpt-full	+35% quality · +306% $/call
summarize_context	80% ✓	= keep gpt-mini	cheapest gate-passer
draft_reply	not evaluated	?	gpt-mini untested on this step
agent_case_brief	69% ✗ quality	↑ gpt-full	+31% quality · +266% $/call
labels	89% ✓	↓ deepseek-v4-flash	-10% quality · -85% $/call

Per-model verdict — strengths, costs, and the blocking gate

One row per candidate. *gate-pass* = steps clearing every gate (quality/schema/reliability/latency). *quality* = mean of per-step avg quality
gate that most often excludes it (where it doesn't pass).

model	verdict	gate-pass	quality	$/call vs gpt	latency vs gpt	main blocker
deepseek-v4-flash	⚠ viable in part	1/10	65%	-95%	1.1×	quality (7/10)
deepseek-v4-pro	⚠ viable in part	3/10	75%	-57%	1.7×	quality (6/10)
gpt-mini	⚠ viable in part	5/10	81%	-71%	0.6×	quality (4/10)
kimi-2.5	❌ unsuitable	0/9	81%	-53%	8.9× !	latency+reliability (9/9)
kimi-2.6	❌ unsuitable	0/10	82%	-4%	6.6× !	reliability (10/10)
phi4	❌ unsuitable	0/9	51%	-100%	3.2× !	reliability (9/9)

appliedAI Developers

Table 1: Metrics Without Complete Ground Truth

How do we measure quality when the golden set is incomplete or anecdotal?

Explore precision versus discovery, partial credit for alternative evidence, known-case performance, novelty, and misleading metrics.

Output: A metric design that rewards valid discoveries without rewarding unsupported claims.

appliedAI institute for europe

appliedAI Developers

Table 2: Evidence & Trace Design

What evidence must the system preserve to make a novel finding verifiable?

Explore source provenance, issue references, conflicting evidence, tool calls, model versions, and storage costs.

Output: A minimum trace specification that supports investigation without overwhelming the infrastructure.

appliedAI institute for europe

appliedAI Developers

Table 3: Experiment & Regression Design

How do we know whether a new agent version actually performs better?

Explore baseline comparisons, sampling, repeatability, changes in discovered patterns, regression detection, and performance on known versus unknown cases.

Output: An experiment that could justify promoting a new version.

appliedAI Developers

Table 4: Trust & Novel Discovery

When should we believe a finding that no human has previously identified?

Explore corroboration, independent verification, conflicting explanations, domain-expert review, confidence, and escalation thresholds.

Output: A validation protocol that distinguishes a hypothesis from a finding strong enough to act upon.

Four things to evaluate

Dimension	What you are checking
Conclusion validity	Is the finding supported?
Evidence quality	Are the cited issues relevant and reliable?
Discovery value	Is the finding novel, useful, and actionable?
System reliability	Can we reproduce, explain, and monitor the finding?

appliedAI Developers

WORLD CAFÉ SYNTHESIS

Dimension	What you are checking	Outcomes
Metrics	What counts as a real improvement?	
Evidence	What must we capture to verify it?	
Experiments	How do we prove it beats the baseline?	
Trust	When does a hypothesis become credible?	

appliedAI institute for europe

appliedAI Developers

Next meetup

Security

Get in touch with us if you want to...

- Host our meetup
- Run a group discussion challenge
- Join as a speaker



Asaad Almutareb
Meetup Moderator & Co-organiser



Kat Jakubowska
Meetup Initiator & Organiser

appliedAI institute for europe